

Reporte

1. Objetivo

Evaluar y mejorar el clasificador de sonidos cardiacos existente (modelo_pcg_final.pkl), que distingue entre grabaciones **Sano**, **Click** y **Soplo** a partir de audio capturado con estetoscopio.

2. Diagnóstico del sistema original

Se revisó el pipeline de producción (arboldeprediccion.py, extract_features.py) y el dataset de entrenamiento (dataset.xlsx, 402 filas). Se encontraron dos problemas antes de intentar cualquier mejora:

- **Sin identificador de origen:** el dataset no tenía ninguna columna que indicara de qué archivo/paciente salía cada fila (cada fila representa un ciclo cardiaco individual, no una grabación completa). Sin esto, es imposible garantizar que la validación del modelo no esté contaminada por ciclos del mismo archivo repartidos entre entrenamiento y prueba.
- **Archivos de audio duplicados:** al agregar el identificador y analizar los 46 audios fuente (biblioteca de la Universidad de Michigan), se descubrió que **44 de los 46 archivos eran copias exactas de solo 9 grabaciones distintas** (confirmado por features idénticas a 6 decimales entre archivos con nombres diferentes). Esto explica por qué evaluaciones previas podían parecer artificialmente perfectas.

3. Búsqueda de datos adicionales

Dado que la biblioteca de Michigan es un recurso de enseñanza con muy pocas grabaciones únicas (~23 en total), se identificó y utilizó el **CirCor DigiScope Phonocardiogram Dataset** (PhysioNet, usado en el Challenge 2022 de George B. Moody): 5,282 grabaciones clínicas reales de 1,568 pacientes, con etiqueta de soplo (presente/ausente) verificada por cardiólogos.

- Se procesaron ~3,007 archivos con el pipeline de extracción existente.
- Se aplicó un filtro de calidad (mínimo 5 ciclos detectados por grabación), quedando **1,079 archivos utilizables de 586 pacientes** (459 Sano, 127 Soplo).
- **Nota importante:** ninguna fuente pública consultada (CirCor, PhysioNet 2016, la propia biblioteca de Michigan) tiene la clase "Click" etiquetada en volumen — sigue existiendo solo 1 grabación única de esa clase. Se recomienda tratarla

como categoría secundaria hasta conseguir más datos, y discutir el alcance con el asesor de servicio social.

4. Metodología de modelado

Se reconstruyó el pipeline paso a paso (preprocesamiento → segmentación → extracción de features → modelado → validación), usando el mismo procesamiento de señal ya existente en el proyecto (envolvente de energía de Shannon + filtro pasabajos, MFCCs con `python_speech_features`).

Punto central de la validación: todas las evaluaciones se hicieron con `StratifiedGroupKFold`, agrupando por paciente — garantizando que ningún paciente apareciera simultáneamente en entrenamiento y prueba (fuga de datos), confirmado explícitamente en cada corrida (0 pacientes repetidos entre lados).

Comparación de algoritmos (5-fold, agrupado por paciente)

Modelo	Exactitud
Regresión Logística (modelo actual)	67.3%
SVM	70.8%
Random Forest	79.1%
Gradient Boosting	79.1%

Se eligió **Random Forest** por su desempeño y por ser más robusto que Regresión Logística ante relaciones no lineales entre features — relevante dado que la segmentación real es imperfecta (ver sección 6).

Selección del umbral de decisión

En vez de usar el umbral por default (0.5), se exploró el trade-off completo entre sensibilidad y especificidad, y se seleccionó el punto óptimo con el **índice de Youden** ($J = \text{sensibilidad} + \text{especificidad} - 1$), tanto a nivel de ciclo individual como a nivel de grabación completa (votación por mayoría entre los ciclos de un mismo audio, igual que hace el sistema en producción).

Validación honesta (dev/test)

Para evitar incluso el sesgo sutil de elegir el umbral óptimo mirando los mismos datos que luego se reportan como resultado, se separó un **20% de los pacientes (117) que**

nunca se usaron ni para entrenar ni para elegir el umbral, exclusivamente como examen final.

Resultado del examen final, con pacientes nunca vistos por el modelo:

- Sensibilidad: **73.5%**
- Especificidad: **69.9%**
- Umbral de decisión final: **0.15**

Este resultado es consistente con la validación cruzada, confirmando que el modelo generaliza como se esperaba y no hay sobreajuste oculto.

5. Integración a producción

Se reemplazó el modelo de PyCaret (3 clases) por el Random Forest de sklearn (2 clases: Sano/Soplo) dentro de arboldeprediccion.py, manteniendo toda la lógica existente de segmentación, votación por mayoría y formato de respuesta para el frontend. El cambio se probó ejecutando el script real (no solo en notebook) contra 30 pacientes de CirCor con etiqueta conocida, confirmando resultados consistentes con la validación formal.

Archivos generados:

- modelo_pcg_soplo_rf.joblib — modelo entrenado con el 100% de los 586 pacientes.
- modelo_pcg_soplo_rf_metadata.json — columnas de features, umbral de decisión, y métricas del examen final documentadas.

6. Limitaciones conocidas

- **Clase "Click" sin datos suficientes** para entrenar o validar (1 grabación única disponible). No se intentó predecirla en el modelo nuevo.
- **La segmentación (detect_cycles) pierde muchos ciclos en audio real y ruidoso.** En grabaciones limpias de enseñanza detecta 70-100 ciclos; en grabaciones clínicas reales de CirCor, la mediana fue de solo 3 ciclos por archivo, y se observaron casos con 1 solo ciclo detectado en grabaciones de 18-26 segundos (donde deberían detectarse 15-25). Esto probablemente le pone un techo al desempeño del modelo, y hace que el BPM y el "porcentaje de confianza" reportados no sean confiables cuando se detectan muy pocos ciclos.

- El modelo se entrenó y validó exclusivamente con CirCor (grabaciones clínicas reales con ruido de fondo real) — no se mezcló con la biblioteca de Michigan (grabaciones de enseñanza, limpias) para evitar que el modelo aprenda a distinguir la *fente* de la grabación en vez de la condición médica.

7. Recomendaciones para siguientes pasos

1. Mejorar detect_cycles para que funcione mejor con audio ruidoso real (por ejemplo, ajustando el umbral de área de los triángulos, o filtrando frecuencias típicas de voz antes de segmentar) — es la mejora con más potencial de subir el desempeño del sistema completo.
2. Agregar una alerta de "confianza baja" cuando se detecten muy pocos ciclos, en vez de reportar 100% de confianza de forma engañosa.
3. Buscar o generar más ejemplos de la clase Click antes de intentar reincorporarla.
4. Usar la biblioteca de Michigan como conjunto de prueba adicional ("¿el modelo sigue funcionando bien en audio limpio?"), no como datos de entrenamiento.